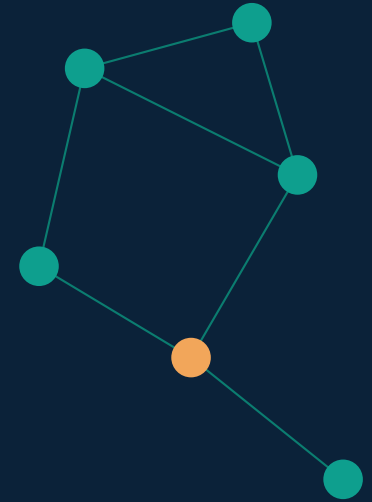


GOOGLE CLOUD ARCHITECTURE

에이전틱 AI 시스템 설계 패턴 선택

워크로드 특성에 맞는 단일·멀티 에이전트 아키텍처 고르기

스터디 발표 자료 · 출처: Google Cloud Architecture Center (Last reviewed 2026-05-28)



AGENDA

오늘 다룰 내용

01

설계 프로세스 & 요구사항

요구사항 정의 → 패턴 검토 → 선택의 3단계와 4가지
평가 축

02

단일 에이전트 시스템

모델·도구·시스템 프롬프트로 동작하는 가장 기본 패턴

03

멀티 에이전트 패턴 8종

순차·병렬·루프·리뷰·비평·반복적 개선·코디네이터·계층·스킴

04

ReAct · 사람 개입 · 커스텀 · 선택 비교

추론·행동 루프, 사람 개입, 커스텀 로직, 그리고 패턴
선택 기준표

CONCEPT

에이전트 AI와 설계 패턴이란

설계 패턴이란?

에이전트 애플리케이션을 만드는 **공통 아키텍처 접근 방식**. 시스템 구성요소를 조직하고, 모델을 통합하고, 단일·다중 에이전트를 오케스트레이션하는 명확한 프레임워크를 제공한다.

- 자율적 의사결정 + 다단계 워크플로 관리
- 외부 데이터로 실시간 문제 해결
- 지식 집약적 작업의 자동화에 강점

에이전트가 필요한가?

✓ 에이전트 적합

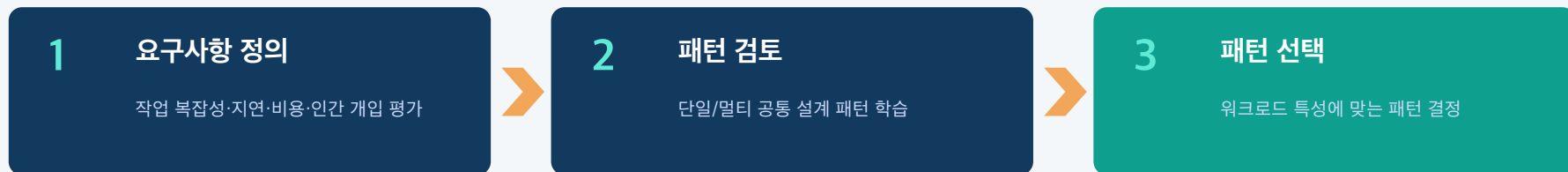
개방형 문제, 자율성이 필요한 목표 지향 작업. 도구를 스스로 쓰고 다단계 계획을 실행해 답을 합성해야 할 때.

— 비에이전트로 충분

예측 가능·고도로 구조화되어 모델 단일 호출로 끝나는 작업(요약·번역·분류)은 에이전트 워크플로가 불필요 — 비용 효율적.

PROCESS

설계 프로세스와 요구사항 정의



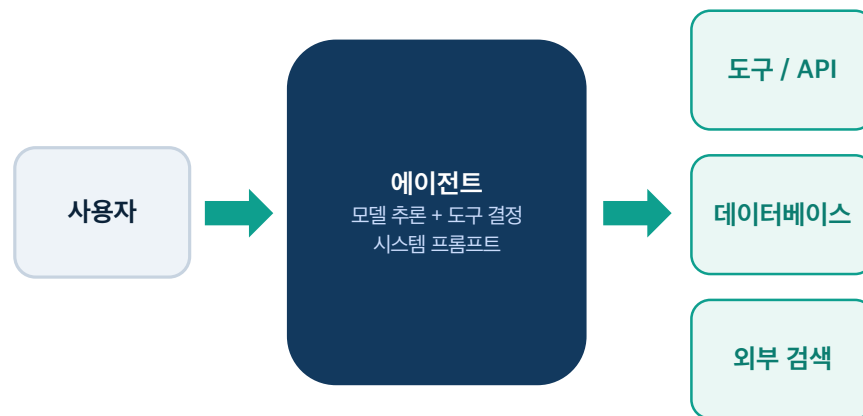
🔄 **일회성 결정이 아니다** — 워크로드·요구사항 변화, 새 기능 출시 때마다 주기적으로 재검토



SINGLE AGENT

단일 에이전트 시스템

동작 구조



정의

하나의 AI 모델 + 정의된 도구 집합 + 포괄적 시스템 프롬프트로 요청을 자율 처리하는 가장 기본 패턴. 에이전트 개발 초기엔 여기서 시작 권장.

사용 사례

다단계·외부 데이터 접근이 필요한 작업 — 주문 상태 DB 조회, API 호출로 최신 뉴스 요약 등.

트레이드오프

도구가 많아지고 복잡해지면 지연·오선택으로 성능 저하 → ReAct로 추론 개선, 그래도 부족하면 멀티 에이전트로.

멀티 에이전트 시스템 개요

큰 목표를 **작은 하위 작업으로 분해**하고, 각 작업을 특정 기술을 가진 전담 에이전트에 위임. 모듈식 설계로 확장성·안정성·유지보수성을 높인다.

DECOMPOSE

핵심 원칙

분해 & 위임 — 협력적 또는 계층적 워크플로우로 상호 작용하여 최종 목표 달성

CONTEXT

컨텍스트 엔지니어링

각 에이전트에 필요한 컨텍스트 관리: 격리, 다단계 유지, 대량 데이터 압축

GOVERNANCE

추가 고려사항

정밀 접근 제어 · 신뢰할 수 있는 에이전트 간 통신 · 다중 실행 비용 관리

이번 발표에서 다루는 멀티 에이전트 패턴 8종

순차

병렬

루프

리뷰·비평

반복적 개선

코디네이터

계층적 분해

스웜

SEQUENTIAL

순차 패턴

동작 구조

사전 정의된 선형 순서



출력 → 다음 입력 (모델 오케스트레이션 불필요)

정의

한 에이전트의 출력이 다음 에이전트의 직접 입력이 되는, 사전 정의된 선형 순서로 전문 에이전트를 실행. 모델 없이 정해진 로직으로 동작.

사용 사례

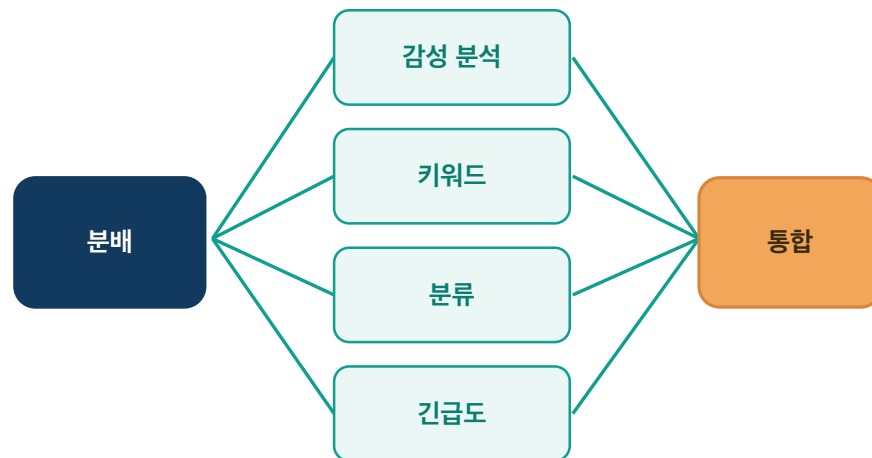
순서가 바뀌지 않는 고도로 구조화·반복 가능한 프로세스 — 데이터 추출 → 정제 → 적재 파이프라인.

트레이드오프

모델 오케스트레이션 대비 지연·비용 낮음. 단, 경직된 구조라 동적 조건 적응·단계 건너뛰기가 어렵다.

PARALLEL 병렬 패턴

동작 구조



정의

여러 전문 하위 에이전트가 동시에 독립적으로 작업하고, 출력을 종합해 최종 응답을 만든다. 모델 없이 병렬 워크플로 에이전트가 조율.

사용 사례

동시 실행으로 지연을 줄이거나 다양한 관점을 모을 때 — 고객 피드백을 감성·키워드·분류·긴급도로 동시 분석.

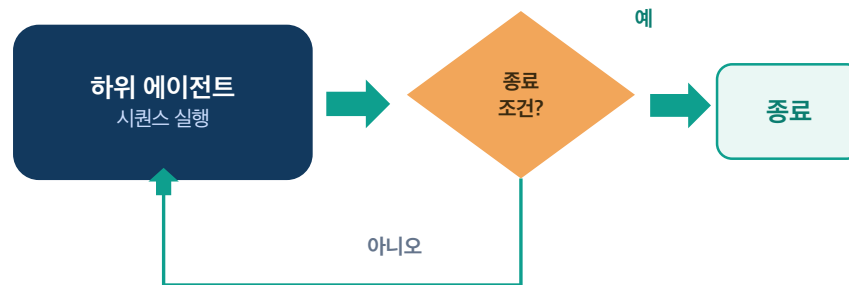
트레이드오프

순차 대비 전체 지연↓. 단, 동시 실행으로 토큰·비용↑, 상충될 수 있는 결과를 합치는 종합 로직이 복잡.

LOOP

루프 패턴

동작 구조



종료 조건 충족까지 하위 에이전트 시퀀스를 반복 실행

정의

종료 조건이 충족될 때까지 하위 에이전트 시퀀스를 반복 실행. 워크플로 에이전트가 모델 없이 동작하며, 매 반복 후 종료 조건(최대 반복 횟수·커스텀 상태)을 평가한다.

사용 사례

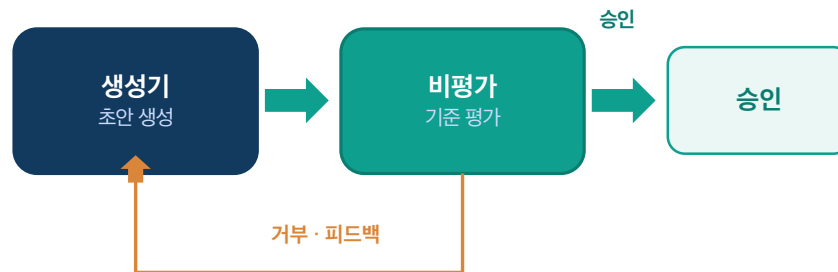
반복적 개선·자기 수정이 필요한 작업 — 콘텐츠를 생성하고 비평 에이전트가 품질 기준을 충족할 때까지 검토.

트레이드오프

복잡한 반복 워크플로 구축 가능. 단 종료 조건이 잘못 정의되면 무한 루프 → 과도한 비용·자원 소비·시스템 행(hang) 위험.

리뷰·비평 패턴 (생성-비평)

동작 구조



생성기 ↔ 비평가: 정확성·형식·안전 기준 충족까지 수정

정의

생성기와 비평가 두 에이전트로 출력 품질·신뢰성을 높이는 루프 패턴 구현. 생성기가 초안을 만들면 비평가가 사전 정의 기준으로 평가해 승인·거부·피드백을 반환한다.

사용 사례

출력이 매우 정확하거나 엄격한 제약을 따라야 하는 작업 — 생성기가 코드를 작성하면 비평가가 보안 감사관으로 취약점 스캔·단위 테스트 통과를 검증.

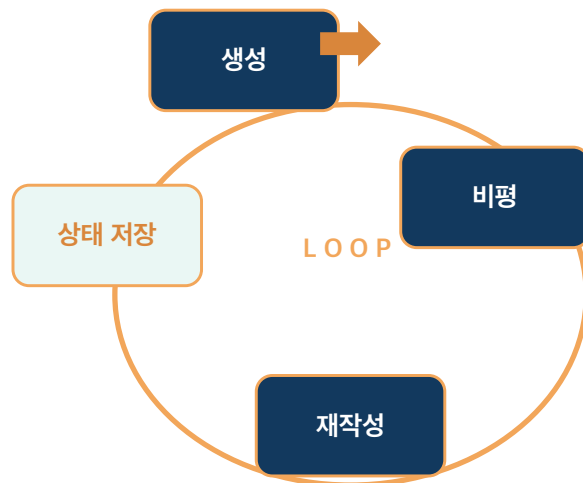
트레이드오프

전용 검증 단계로 품질·정확성·신뢰성↑. 단 비평가 평가에 최소 1회 추가 모델 호출, 수정 루프가 반복되면 지연·비용 누적.

반복적 개선 패턴

★ 핵심 — URL 지정 패턴

동작 구조



품질 임계값 충족 또는 최대 반복 도달 시 종료

정의

루프 메커니즘으로 출력을 여러 사이클에 걸쳐 점진적으로 개선. 루프 패턴의 구현체로, 세션 상태의 결과를 매 반복마다 수정한다.

사용 사례

한 번에 완성하기 어려운 복잡한 생성 작업 — 코드 작성·디버깅, 다단계 계획 수립, 장문 문서 초안→비평→재작성.

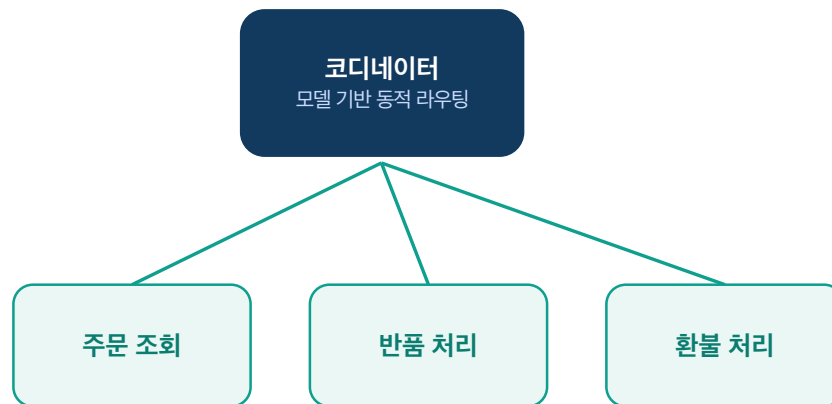
트레이드오프

고품질·정교한 결과 가능. 단 사이클마다 지연·비용↑. 무한 루프 방지를 위한 종료 조건(품질 평가·최대 반복) 설계가 필수.

코디네이터 패턴

동작 구조

AI 모델이 요청을 분석·분해해 분기



정의

중앙 코디네이터가 요청을 분석·분해해 각 하위 작업을 전문 에이전트에 동적으로 라우팅. 병렬 패턴과 달리 AI 모델로 오케스트레이션.

사용 사례

적응형 라우팅이 필요한 구조화된 업무 자동화 — 고객 서비스가 주문/반품/환불을 판별해 적절한 에이전트로 분기. (계층적 분해는 이 패턴의 확장)

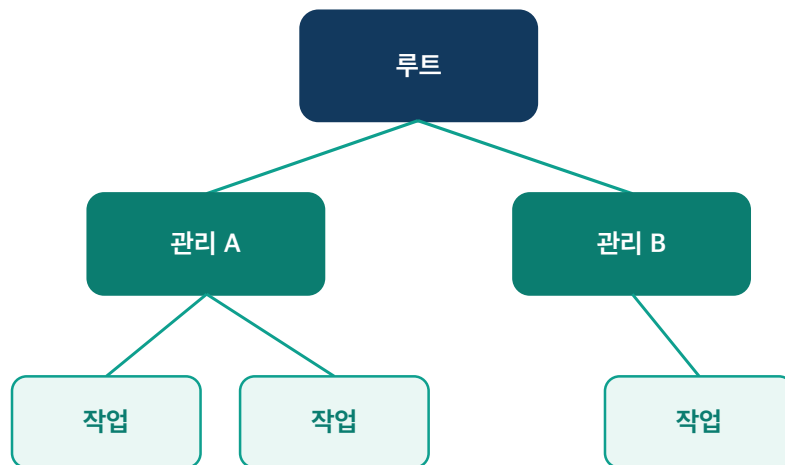
트레이드오프

유연성↑. 단 코디네이터+전문 에이전트 모두 모델에 의존해 호출 수·토큰·비용·지연이 단일 에이전트보다 증가.

HIERARCHICAL

계층적 작업 분해 패턴

동작 구조



루트 → 하위 코디네이터 → 워커: 다단계로 점진 분해

정의

에이전트를 다단계 계층으로 조직하는 코디네이터 패턴의 구현. 최상위 루트 에이전트가 복잡한 작업을 작은 하위 작업으로 분해해 하위 전문 에이전트에 위임하며, 이 과정이 여러 계층에 걸쳐 반복된다.

사용 사례

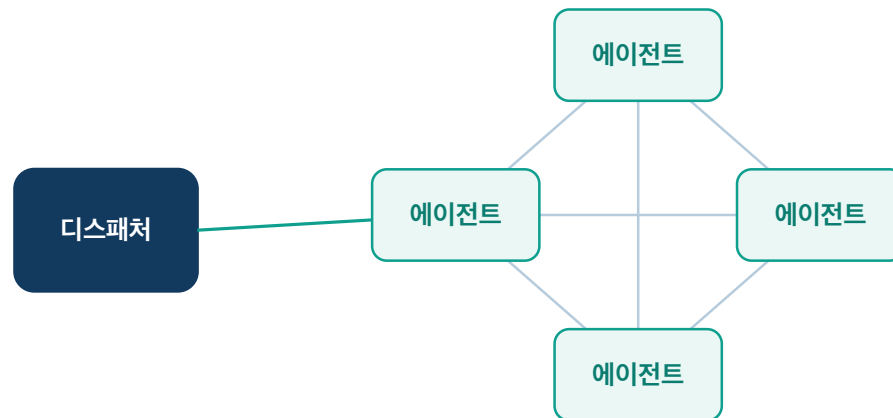
다단계 추론이 필요한 모호·개방형 문제 — 연구 프로젝트에서 코디네이터가 정보 수집·분석·보고서 작성으로 분해해 각 전문 하위 에이전트에 위임.

트레이드오프

복잡·모호 문제를 체계적으로 분해해 포괄적·고품질 결과. 단 다단계 구조로 설계·디버깅·유지보수 어렵고, 다수 모델 호출로 지연·비용이 크게 증가.

SWARM 스웜 패턴

동작 구조



중앙 조율자 없이 에이전트 간 전방위 통신으로 협력·수렴

정의

협력적 전방위(all-to-all) 통신 방식. 디스패처가 요청을 적합한 에이전트로 전달하되 오케스트레이션은 하지 않고, 각 에이전트가 서로 결과 공유·비평·핸드오프하며 해를 반복 개선한다. 명시적 종료 조건이 필수.

사용 사례

토론과 반복 개선이 유익한 모호·고복잡 문제 — 신제품 설계에 시장조사·엔지니어링·재무 모델링 에이전트가 아이디어를 공유하고 트레이드오프를 토론했을 때 최종 사양에 수렴.

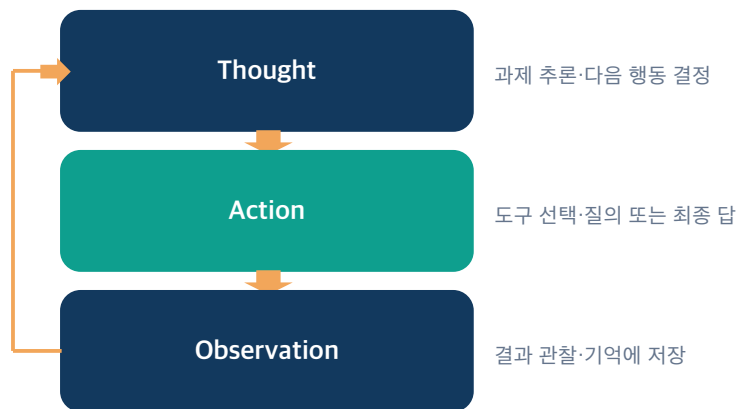
트레이드오프

전문가 팀 협업을 시뮬레이션해 탁월·창의적 결과. 단 가장 복잡·고비용이며, 조율자 부재로 비생산 루프·수렴 실패 위험이 있어 정교한 통신 제어가 필요.

ReAct

추론-행동(ReAct) 패턴

동작 구조



종료 조건 충족까지 Thought-Action-Observation 반복

정의

AI 모델이 사고 과정과 행동을 자연어 상호작용으로 구성. Thought → Action → Observation 루프를 종료 조건까지 반복한다.

사용 사례

지속적 계획·적응이 필요한 복잡·동적 작업 — 새 장애물을 피하며 경로를 갱신하는 로봇틱스 경로 생성 등. 단일 에이전트에 적용 가능.

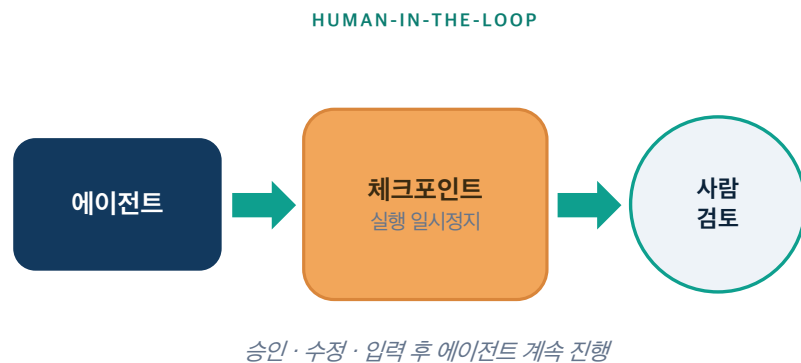
트레이드오프

추론 기록으로 디버깅 용이, 단순·저비용. 단 다단계 루프로 지연↑, 한 단계의 잘못된 관찰이 최종 답까지 전파될 수 있음.

HUMAN-IN-THE-LOOP

사람 개입 패턴

동작 구조



정의

워크플로 내 사전 정의된 체크포인트에서 에이전트가 실행을 멈추고 사람의 검토를 기다린다. 승인·오류 수정·필요 입력 후 진행.

사용 사례

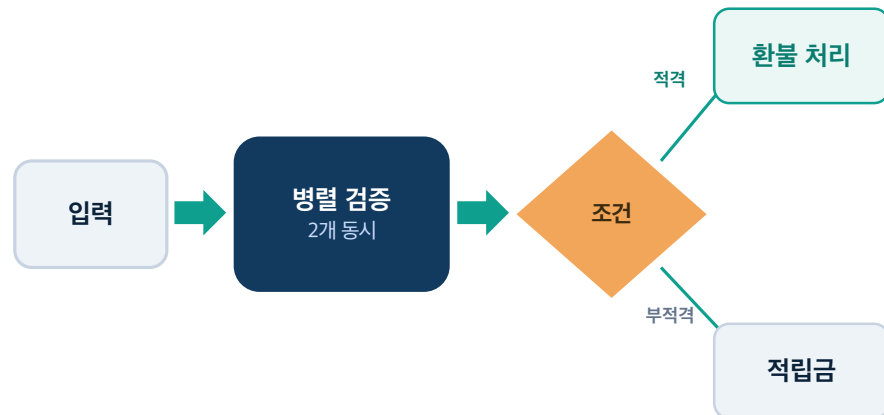
사람의 감독·주관적 판단·최종 승인이 필요한 작업 — 거래 거래 승인, 민감 문서 요약 검증, 환자 데이터 익명화의 컴플라이언스 승인.

트레이드오프

중요 결정에 사람 판단을 삽입해 안전·신뢰성↑. 단 사용자 상호작용용 외부 시스템 구축·유지로 아키텍처 복잡도 증가.

커스텀 로직 패턴

동작 구조



병렬·조건 분기 등 패턴을 혼합한 맞춤 오케스트레이션

정의

워크플로 설계에 최대 유연성을 제공. 조건문 등 코드로 직접 오케스트레이션 로직을 구현해 여러 분기 경로를 가진 복잡한 워크플로를 만든다.

사용 사례

세밀한 실행 제어가 필요하거나 다른 패턴에 맞지 않는 워크플로 — 환불 처리에서 병렬 검증 후 조건 분기로 적격 시 환불, 부적격 시 적립금 흐름으로 라우팅.

트레이드오프

세밀한 제어 가능. 단 전체 오케스트레이션을 직접 설계·구현·디버깅해야 해 개발·유지보수 복잡도와 오류 가능성↑ (ADK의 사전 정의 패턴 대비).

DECISION GUIDE

패턴 선택 — 워크로드 특성별 가이드

결정적 워크플로

예측 가능·순차적, 시작-끝 경로가 명확

- 고정된 단계, 모델 오케스트레이션 불필요 **순차 패턴**
- 동시 실행 가능한 독립 작업 **병렬 패턴**
- 한 번에 어려운 생성, 점진적 품질 개선 **반복적 개선 패턴**

반복 워크플로

개선·피드백 사이클로 결과 완성

- 추론-행동-관찰로 계획 구축·적응 **ReAct 패턴**
- 종료 조건까지 정해진 동작 반복 **루프 패턴**
- 완료 전 별도 검증 단계 필요 **리뷰·비평 패턴**

동적 오케스트레이션

에이전트가 진행 방식을 스스로 판단·조율

- 외부 도구 + 빠른 프로토타입 **단일 에이전트**
- 입력별 동적 라우팅이 필요 **코디네이터 패턴**
- 복잡·모호, 다수준 분해가 핵심 **계층적 분해 패턴**

특수 요구사항

공통 패턴에 맞지 않는 맞춤 로직

- 고위험·주관·컴플라이언스 감독 필요 **사람 개입 패턴**
- 분기 많은 복잡 로직, 최대 제어 **커스텀 로직 패턴**
- 다관점 종합·협력적 토론 **스웸 패턴**

KEY TAKEAWAYS

정리 — 기억할 4가지

1

단일 에이전트부터 시작

핵심 로직·프롬프트·도구 정의를 다듬고, 부족할 때 멀티로 확장

2

워크플로의 '결정성'으로 갈래를 나눠라

고정 순서면 워크플로 에이전트(순차·병렬), 모델 판단이 필요하면 코디네이터·ReAct

3

모든 선택은 트레이드오프

품질·유연성 ↔ 지연·비용·복잡도. 모델 호출이 늘수록 비용과 지연이 커진다

4

일회성 결정이 아니다

요구사항·워크로드 변화와 새 기능에 맞춰 아키텍처를 주기적으로 재검토

■ REFERENCES

참고 자료

Google Cloud — 에이전틱 AI 시스템의 설계 패턴 선택

docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system?hl=ko

Google Cloud Architecture Center · 본 발표의 원문 (한국어)